参考答案（书末集中，先自己想再看）

每题四步式：① 先想什么 ② 为什么用这个命令 ③ 完整步骤 ④ 怎么验证做对了。

关于示例输出：示例里的**路径、PID、时间、版本号、空格/对齐**会因系统版本和用户名而不同——**看结构就行**（例如 `uniq -c` 的计数列在多数系统上右对齐、前面带空格），不必逐字对齐。

阶段 1 答案

【基础 1-1 ★】多级目录

① **先想什么**：mkdir 默认不会自动补出中间的父目录（并列的多个目录它可以一次建），a/b/c 是三层嵌套，需要一次性建完的选项。

② **为什么用这个命令**：`mkdir -p` 的 p 是 parents（父级）——目标路径里不存在的父目录会自动先建。没有 -p 会报"没有那个文件或目录"。

③ **完整步骤**：

```
$ mkdir -p ~/lab1/a/b/c
$ ls -R ~/lab1        # -R 递归显示，能看到 lab1/a、lab1/a/b、lab1/a/b/c
```

④ **怎么验证做对了**：`ls -R ~/lab1` 能看到三层目录嵌套出现；再试 `mkdir ~/lab1/x/y`（不带 -p），应得到报错 `mkdir: No such file or directory`（**老版本**会写成 `mkdir: cannot create directory '…'`，26.04 已简化）——这证明 -p 是必需的。

【基础 1-2 ☆】绝对/相对路径

① **先想什么**：在 /tmp 里建目录，目标却想放在家目录——跨目录操作，要看"从哪出发描述目标"。

② **为什么用这个命令**：绝对路径从 / 写起（`/home/你的真实用户名/lab2`），在任何位置都指向同一处；相对路径从当前位置算起（`lab2` 指 /tmp/lab2，`~/lab2` 指家目录，~ 是家目录的缩写）。此题要写出真正的绝对路径：`/home/你的真实用户名/lab2`（用户名用 whoami 查）。注意：~ 只是家目录的缩写，不是绝对路径写法——绝对路径必须从树根 / 出发。

③ **完整步骤**：

```
$ cd /tmp
$ mkdir -p ~/lab2/a/b/c      # 家目录版（可照抄，~ 自动展开）
$ ls /home/                  # 观察：能看到你的真实用户名目录——~/lab2 的真实绝对路径就是 /home/你的真实用户名/lab2
$ mkdir -p lab2-tmp/a/b/c    # 相对路径：建在当前目录，即 /tmp/lab2-tmp
$ ls -d ~/lab2 /tmp/lab2-tmp   # 两个位置各有一棵目录树
```

④ **怎么验证做对了**：在 /tmp 下执行 `pwd` 确认位置后，分别用相对和绝对路径创建，观察两个结果各落在哪里——这题的目的就是让你"踩一次"路径错乱的坑。

【基础 1-3 ★】批量创建

① **先想什么**：五个文件，一条命令——需要"一次写多个名字"的机制。

② **为什么用这个命令**：bash 的花括号展开 `{1..5}` 会生成 1、2、3、4、5 的序列，所以 `file{1..5}.txt` 展开成 file1.txt … file5.txt，touch 一次收到 5 个名字。

③ **完整步骤**：

```
$ cd ~/lab1
$ touch file{1..5}.txt
$ ls
```

④ **怎么验证做对了**：`ls` 应看到 file1.txt 到 file5.txt 整齐列出。想看清展开过程，可先跑 `echo file{1..5}.txt`——echo 会把展开结果打印出来，这招以后验证"shell 怎么理解你的命令"时经常用。

【基础 1-4 ★】cp 递归

① **先想什么**：目录里还有子内容，cp 会怎么处理"里面那一堆"？

② **为什么用这个命令**：cp 默认只复制文件，遇到目录直接拒绝（防止把一大棵树意外复制错）。**`-r`**（recursive，递归）就是告诉它"整个目录连里面的东西一起复制"。

③ **完整步骤**：

```
$ cd ~                      # 先回到家目录：1-3 结束时你在 ~/lab1 里，而 lab1 在 ~ 下
$ cp lab1 lab1_copy         # 故意不带 -r
cp: -r not specified; omitting directory 'lab1'
$ cp -r lab1 lab1_copy      # 带上 -r 就成功
```

④ **怎么验证做对了**：看到 `omitting directory`（省略目录）的报错并理解了它，本题即算完成——你记住了"复制目录永远要 -r"。

【基础 1-5 ☆】cp 两种语义

① **先想什么**：cp 的目标参数是"目录"还是"新名字"，结果完全不同。

② **为什么用这个命令**：cp 有两条规则——① 目标是**已存在的目录**：复制进去，文件名不变；② 目标是**不存在的名字**：复制并改成这个名字。一条命令同时完成了 Windows 里的"复制 + 粘贴"两步。

③ **完整步骤**：

```
$ cd ~/lab1                   # 1-4 结束时你在家目录，先回到练习目录
$ mkdir -p backup         # 先建目标目录（空目录也能跑完整示范）
$ cp file1.txt backup/        # backup 已存在 → 里面出现 file1.txt
$ cp file1.txt file1_bak.txt # 新名字 → 当前目录多出 file1_bak.txt
$ ls backup/; ls
```

④ **怎么验证做对了**：两种操作后，一个出现在 backup/ 里且名字不变，一个出现在当前目录且换了名字。用一句话总结：**目标是目录就"放进去"，目标不是目录就"复制并改名"。**

【基础 1-6 ★】mv 双功能

① **先想什么**：搬家和改名其实是同一件事——改变文件在目录树里的"地址"。

② **为什么用这个命令**：mv（move）改的是文件的"路径"：路径的目录部分变了=搬家，名字部分变了=改名。一步完成"搬到新目录 + 起新名字"。

③ **完整步骤**：

```
$ cd ~/lab1
$ printf '第一行\n第二行\n这一行里有关键词\n' > file1.txt   # 先写入三行内容（第三行含"关键词"，供 2-6 递归搜索用）
$ mv file1.txt ~/lab1/a/renamed.txt   # 搬家 + 改名，一步
$ ls . ~/lab1/a/        # 原位置消失，新位置出现 renamed.txt
```

④ **怎么验证做对了**：先 `touch file1.txt` 重建一个（它已被 mv 搬走），再对比 `cp file1.txt file1-copy-demo.txt` 做一次——cp 后原文件还在，mv 后原位置空了。（用新名字对比，避免覆盖 2-6 要用的 renamed.txt）这就是本质区别：**cp 是"复印"，mv 是"搬走"。**

【基础 1-7 ★】删除权限原理

① **先想什么**：直觉以为"删除文件 = 改文件"，但删除时系统改的其实是**文件所在目录。**

② **为什么用这个命令**：文件在文件系统里只是目录里的一条"登记信息"（名字 → 数据位置）。删除 = 把这条登记信息从目录里划掉，所以需要的是**目录的写权限**，与文件本身的权限无关。只读文件照样能删（rm 会问一句确认）。

③ **完整步骤**：

```
$ touch readonly.txt
$ chmod 444 readonly.txt    # 设成只读：r--r--r--
$ ls -l readonly.txt
$ rm readonly.txt           # 普通用户 + 交互终端：会提示 remove write-protected ... ? 回答 y（root/sudo 或非交互下不提示，直接删）
y → 删除成功
```

④ **怎么验证做对了**：只读文件被删掉了。反向验证：把文件放进一个**不可写的目录**，即使文件本身 777 也删不掉（会报 Permission denied）——删不删得掉，看目录，不看文件。

【基础 1-8 ☆】安全删除

① **先想什么**：rm 不可恢复，所以真正的考点是"删之前的确认习惯"。

② **为什么用这个命令**：`rm -r` 递归删整棵树；`rm -i`（interactive）每删一个文件都先问 y/n——高危操作前用 -i 过一遍，是运维的基本素养。

③ **完整步骤**：

```
$ mkdir -p ~/lab1/tree/x/y
$ touch ~/lab1/tree/x/y/f1 ~/lab1/tree/x/f2
$ rm -ri ~/lab1/tree      # 每个文件都会问 y/n，逐个确认
$ ls ~/lab1/tree          # No such file or directory → 整棵树已删
```

④ **怎么验证做对了**：-i 模式下每个文件都停了问你，你回答 y 才继续；整个过程删得干净且全程可控。养成习惯：**大范围删除先 ls 看清单，再用 -i 确认，最后才敢用裸 rm -r。**

阶段 2 答案

【基础 2-1 ★】创建与重定向

① **先想什么**：键盘输入的默认去向是终端屏幕，想让它"流进文件"，要用重定向。

② **为什么用这个命令**：`cat > file` 让 cat 从键盘读、把输出写进 file（而不是屏幕）；输入完按 Ctrl+D 表示"输入结束"。

③ **完整步骤**：

```
$ cat > ~/lab1/note.txt
你好，Linux
这是第二行
按 Ctrl+D
$ echo "覆盖" > ~/lab1/note.txt
$ echo "追加" >> ~/lab1/note.txt
$ cat ~/lab1/note.txt
```

④ **怎么验证做对了**：最后 cat 看到的只有两行——"覆盖"和"追加"。原来的"你好，Linux"被 > 清掉替换了；"追加"是 >> 接在后面的。一个文件、两次写入、三种结局，全看清了。

【基础 2-2 ☆】覆盖 vs 追加

① **先想什么**："覆盖"到底先清空还是直接写？设计一个实验让它现原形。

② **为什么用这个命令**：> 打开文件时会把文件**截断到 0 字节**再写；>> 则打开后直接写到末尾。所以新内容比旧内容短时，> 的结果里不会残留旧尾巴。

③ **完整步骤**：

```
$ echo "12345678" > test.txt
$ echo "abc" > test.txt
$ cat test.txt        # abc（不是 abc45678！）
$ echo "abc" >> test.txt
$ cat test.txt        # 两行 abc（旧的 abc 还在）
```

④ **怎么验证做对了**：第一次 > 写完后只剩 abc，证明旧的 8 个字符被**先清空**了；第二次 >> 后变成两行 abc，证明追加不动旧内容。若 > 只是"从头写"而不清空，你会看到 abc45678——实验结果排除了这个可能。

【基础 2-3 ★】头尾查看

① **先想什么**：大文件不能 cat 全看，只想要开头和结尾。

② **为什么用这个命令**：head 默认前 10 行、tail 默认后 10 行；`tail -f`（follow，跟随）不退出，文件一有新内容立刻刷出来——看实时日志的唯一姿势。

③ **完整步骤**：

```
$ printf 'line1 ok\nERROR disk slow\nerror in app\nwarn low mem\n' > ~/lab/app.log   # 自建材料：4 行，含 error/ERROR 各一行（供 2-3~2-7 使用；2-4 另有 120 行材料生成法）
$ head ~/lab/app.log
$ tail ~/lab/app.log
$ tail -f ~/lab/app.log    # 另开一个终端执行：echo test >> ~/lab/app.log（追加自建日志，平板同样可用）
# 观察本终端立刻刷出新行，Ctrl+C 退出
```

④ **怎么验证做对了**：另开的终端里写入一行后，-f 这边马上多出这一行——这就是"实时跟踪"。排障时开着 tail -f 看日志，复现一次故障，根因自己蹦出来。

【基础 2-4 ☆】指定行数

① **先想什么**：head/tail 都只能从"边"上取，中间一段要两次裁剪拼出来。

② **为什么用这个命令**：先 head 取前 120 行，再 tail 取这 120 行里的最后 21 行，正好是第 100~120 行。`tail -n +100` 表示"从第 100 行开始取到末尾"，是另一种解法。

③ **完整步骤**：

```
$ seq 1 120 > ~/lab/lines.log   # 先造材料：seq 生成 1~120 共 120 行（本册第一次出现：seq = 生成数字序列，作用就是快速造出带行号的材料）。用独立的 lines.log，不动 app.log—2-5、2-7 还要用 app.log 里的 error 行
$ head -n 120 ~/lab/lines.log | tail -n +100   # 第100~120行，共21行
# 等价写法：head -n 120 ~/lab/lines.log | tail -n 21
```

④ **怎么验证做对了**：数一下输出行数应是 21 行（`... | wc -l` 验证），且第一行内容与 `head -n 100 ~/lab/lines.log | tail -n 1` 一致（head|tail 第 3 章学过）。这种"两头裁剪取中间"的组合，以后你会反复用到。做完顺手 `rm ~/lab/lines.log` 清理。

【基础 2-5 ★】grep 过滤

① **先想什么**：日志几千行，只关心含 error 的——这是"过滤"。

② **为什么用这个命令**：grep 按行匹配。`-i`（ignore case）忽略大小写；`-v`（invert）反选=要"不含"的行；`-c`（count）只数有几行不显示内容。

③ **完整步骤**：

```
$ grep error ~/lab/app.log
$ grep -i error ~/lab/app.log    # Error/ERROR 也抓到
$ grep -v error ~/lab/app.log    # 不含 error 的行
$ grep -c error ~/lab/app.log    # 只输出一个数字
```

④ **怎么验证做对了**：-c 的数字 ≤ -i 的行数 ≤ 总行数，且 -v 的行数 + 含 error 的行数 = 总行数（grep 逐行匹配，每行要么匹配要么不匹配，等式严格成立）。选项会拼起来用：`grep -ivc` 一次三个。

【基础 2-6 ★】递归搜索

① **先想什么**：知道内容、不知道在哪个文件里，要整棵树搜。

② **为什么用这个命令**：`grep -r`（recursive）对目录递归搜索；`-n` 显示"文件名:行号"，找到后能直接定位。

③ **完整步骤**：

```
$ mkdir -p ~/lab1/a && printf '第一行\n第二行\n这一行里有关键词\n' > ~/lab1/a/renamed.txt
# 若跳过了阶段 1，先自建材料（含"关键词"的第三行）
$ grep -rn "关键词" ~/lab1/
/home/你的真实用户名/lab1/a/renamed.txt:3:这一行里有关键词（输出里的路径以你机器上的实际绝对路径为准）
```

④ **怎么验证做对了**：输出的每一行格式是"文件:行号:内容"，照着路径和行号打开文件能对上。参数顺序自由，`-rn` 与 `-nr` 等价。

【基础 2-7 ★】管道实战

① **先想什么**：过滤完要数数——两步操作一步完成，数据从 grep 直接流进 wc。

② **为什么用这个命令**：| 把左边命令的"标准输出"接到右边命令的"标准输入"。grep 的输出（几百行）不落盘，直接流进 wc 数行数。

③ **完整步骤**：

```
$ grep -i error ~/lab/app.log | wc -l
```

④ **怎么验证做对了**：输出的数字与 `grep -ic error ~/lab/app.log` 的结果一致（两种方法互证）。这是你写的第一个"流水线"：**查看 → 过滤 → 统计**，Linux 的威力从这里开始。

【基础 2-8 ☆】管道 vs 重定向

① **先想什么**：都是"数据的出口"，但目的地不同——一个是命令，一个是文件。

② **为什么用这个命令**：| 把数据交给**下一个命令**继续加工（命令到命令）；> 把数据**写进文件**存起来（命令到文件）。方向错了会报错：文件不会"执行"，命令也不会"接收"重定向来的内容。

③ **完整步骤**：

```
$ printf 'ok line\nerror in app\nwarn only\nERROR big\n' > log.txt   # 自建材料：4 行，含 error、ERROR 各一
$ grep error log.txt | wc -l        # 管道：结果继续被加工
$ grep error log.txt > result.txt  # 重定向：结果存进文件
```

④ **怎么验证做对了**：一句话总结能讲清就算对：**"管道是给下一个命令吃的，重定向是写进文件存的。"** 再试试把两者接起来：`grep error log.txt | wc -l > result.txt`——数据先流经命令加工，最后流进文件，一条龙。

阶段 3 答案

【基础 3-1 ★】权限位解读

① **先想什么**：`-rw-r--r--` 这串字符要拆成"类型 + 三组权限"读。

② **为什么用这个命令**：第一位是类型（- 文件、d 目录），后 9 位按"属主/属组/其他人"三组、每组"读r/写w/执行x"排列。`-rw-r--r--` = 文件，属主 rw-，属组 r--，其他人 r--。

③ **完整步骤**：

```
$ ls -l /etc/hostname      # 找一个文件练手
-rw-r--r-- 1 root root 13 ...
数字法：r=4 w=2 x=1 → 755 = rwxr-xr-x；644 = rw-r--r--
```

④ **怎么验证做对了**：能脱口而出：644 是"我能读写、别人只能读"（最常见文件权限）；755 是"我能读写执行、别人能读能执行"（最常见程序/目录权限）。看到任何 9 位串 3 秒内拆出三组权限即达标。

【基础 3-2 ☆】两种改法

① **先想什么**：一个文件 644，想变成 744——用"加一位"的符号法，还是"整体重写"的数字法?

② **为什么用这个命令**：符号法 **`chmod u+x`** 表示"属主(u)加(+)执行(x)"，其余位不动；数字法 **`chmod 744`** 把三位**整体覆盖**成新值。目标是同一权限时，两种写法结果相同；符号法适合微调，数字法适合全改。

③ **完整步骤**：

```
$ touch f; chmod 644 f
$ chmod u+x f; ls -l f      # -rwxr--r--（只给属主加了一位 x）
$ chmod 644 f; chmod 744 f; ls -l f    # 也是 -rwxr--r--
```

④ **怎么验证做对了**：两次结果都是 `-rwxr--r--`。再体会一个坑：如果文件原本 600，用 `chmod u+x` 得到 700（其他位保留），而 `chmod 744` 会把其他位也变成 r--——**数字法是"整体覆盖"，不是"只加一位"**，这是两者最大的不同。

【基础 3-3 ★】建用户

① **先想什么**：建账号是动系统的操作，需要管理员权限。

② **为什么用这个命令**：**`useradd -m`** 建用户并生成家目录（-m = make home）；**`passwd`** 设密码。都要 sudo。

③ **完整步骤**：

```
$ getent passwd alice bob    # 冲突检查：无输出=可以创建；有输出=上次残留，先清理（sudo userdel -r 对应账号）再继续，或停止本阶段实验
$ sudo useradd -m -s /bin/bash alice    # -m 建家目录；-s 指定登录 shell（与教科书 5.2 一致）
$ sudo passwd alice          # 按提示输两遍密码
$ sudo useradd -m -s /bin/bash bob
$ sudo passwd bob
$ ls /home                   # alice、bob 的家目录都出现了
```

④ **怎么验证做对了**：`ls /home` 出现两个新目录；`id alice` 能看到她的 uid/gid。用完可以清理：`sudo userdel -r alice`（-r 连家目录一起删）。

【基础 3-4 ★】越权验证

① **先想什么**：先记录 bob 家目录原权限，并确保执行访问命令的身份明确。

② **为什么用这个命令**：目录权限决定能否进入；先把前提设置为 700，再用 `sudo -u alice` 明确以 alice 身份测试，避免交互式会话切换位置不清。

③ **完整步骤**：

```
$ original_mode=$(stat -c %a /home/bob) # stat 本册第一次出现：查看文件详细状态；-c %a = 只输出"权限数字"（%a 即八进制权限）。验证对照：stat -c %a 与 ls -l 的 9 位权限互相对得上（如 -rwxr-xr-x ⇔ 755）
$ sudo chmod 700 /home/bob   # 让测试前提可控
$ sudo -u alice sh -c 'whoami; ls /home/bob' # 显示 alice，然后 Permission denied
$ sudo chmod "$original_mode" /home/bob # 恢复原权限
$ ls -ld /home/bob          # 再次核对
```

④ **怎么验证做对了**：输出身份必须是 alice；访问 bob 目录得到 Permission denied；最后权限数字应与 original_mode 相同。若系统没有 alice，先按阶段 3-3 创建练习用户；实验完成后删除练习用户和临时对象。

【基础 3-5 ★】chown 影响

前提：本答案假设 alice 不在该文件的属组内（此时她按"其他人"权限位判断）；若 alice 与文件同属一组，则应按属组权限位判断，结论相应改变。

环境前提（Ubuntu 26.04 实测一致）：新装 Ubuntu（21.04 起）家目录默认是 750（drwxr-x---），alice/bob 穿不过 /home/<你>，会直接 Permission denied。先确认：`ls -ld /home/$(whoami)`——若显示 drwxr-x---，临时 `sudo chmod 711 /home/$(whoami)`（别人"只能穿过、不能列目录"），做完实验务必恢复：`sudo chmod 750 /home/$(whoami)`。

① **先想什么**：文件换了主人后，原主人变成了"其他人"还是"同组"，权限就跟着变。

② **为什么用这个命令**：`chown bob file` 把属主换成 bob。若文件 644（rw-r--r--），alice 不再是属主，只能以"其他人"身份读（r），不能写也不能执行。

③ **完整步骤**：

```
$ cd ~/lab1
$ sudo rm -f file1.txt     # 重复运行本实验先清理旧文件（上次实验后它的属主是 bob，直接 touch 会 Permission denied）
$ touch file1.txt     # 阶段 1 里 file1.txt 已被 mv 改名成 renamed.txt，这里重建一个供本实验用
$ sudo chown bob file1.txt
$ ls -l file1.txt          # 属主已变成 bob
$ sudo -u alice cat "$PWD"/file1.txt      # 以 alice 身份读：能读（其他人有 r）；"$PWD" 自动展开成当前目录真实路径
$ sudo -u alice bash -c 'echo x >> "$PWD"/file1.txt' # 以 alice 身份写：Permission denied（其他人无 w）
$ id # 自始至终没切过会话，身份清晰，不用 exit 退出
```

④ **怎么验证做对了**：读成功、写失败，与你按权限位推演的结果一致。关键理解：**判断权限不是看"我是谁"，而是看"对这个人而言我是属主、属组还是其他人"**——文件换主后，原主人的身份也跟着变了。

【基础 3-6 ☆】组共享

环境前提（Ubuntu 26.04 实测一致）：新装 Ubuntu（21.04 起）家目录默认是 750（drwxr-x---），alice/bob 穿不过 /home/<你>，会直接 Permission denied。先确认：`ls -ld /home/$(whoami)`——若显示 drwxr-x---，临时 `sudo chmod 711 /home/$(whoami)`（别人"只能穿过、不能列目录"），做完实验务必恢复：`sudo chmod 750 /home/$(whoami)`。

① **先想什么**：两个人要能写同一个目录，但又不给"所有人"写——用"属组"这个中间层。

② **为什么用这个命令**：建组 **groupadd**，把人加进组 **usermod -aG**，目录属组设成该组 **chgrp**，组权限给 rwx。这样"组内能写、组外不能"。

③ **完整步骤**：

```
$ getent group shared    # 冲突检查：无输出=可以创建；已存在=先 sudo groupdel shared 清理（确认是本实验建的才删）
$ sudo groupadd shared
$ sudo usermod -aG shared alice
$ sudo usermod -aG shared bob
$ mkdir -p ~/lab/shared   # 专用实验目录放在练习区（~/lab），不动系统目录 /srv
$ sudo chgrp shared ~/lab/shared
$ sudo chmod 2770 ~/lab/shared    # 2=setgid：新文件自动继承组的身份
```

④ **怎么验证做对了**：alice 和 bob 各自登录，都能在 ~/lab/shared 里建文件；而 `ls -ld ~/lab/shared` 显示 `drwxrws---`（中间的 s 就是 setgid）。组外用户进不去。提示：-aG 的 a 是 append（追加），漏掉 a 会把用户从其他组里踢出去——这也是新手常踩的坑。 收尾（练习结束恢复现场）：sudo groupdel shared; sudo rm -rf ~/lab/shared; sudo userdel -r alice; sudo userdel -r bob（userdel 一次只能删一个用户；报 No such user = 已删或没建过，属正常）

【基础 3-7 ★】账号文件

① **先想什么**：用户信息存在哪？直接看系统账号数据库。

② **为什么用这个命令**：/etc/passwd 一行一个用户，7 个冒号分隔字段：**用户名:x(密码占位):UID:GID:描述:家目录:登录shell**。/etc/group 类似：组名:x:GID:成员列表。

③ **完整步骤**：

```
$ grep alice /etc/passwd
alice:x:1001:1001::/home/alice:/bin/bash
$ grep shared /etc/group
shared:x:1002:alice,bob
```

（示例输出：UID / GID 是顺序分配的编号——以你机器为准，只要字段结构对就行。）

④ **怎么验证做对了**：能逐字段说出每个冒号间隔的含义。注意 x 不是密码——密码加密后存在 /etc/shadow（只有 root 能看），这是"账号数据"与"密码数据"分离的安全设计。

【基础 3-8 ☆】删除权限回顾

① **先想什么**：结合 1-7 的结论推演：删除看目录的写权限，那"目录可写"时无论文件是谁的，都能删。

② **为什么用这个命令**：删除 = 从目录里划掉登记项，需要目录 w 权限，跟文件的属主无关。所以"目录可写 + 文件属主是别人"→ 能删。唯一例外是 sticky bit（粘滞位，/tmp 就有，权限位显示 t）：此时只有文件属主或 root 能删。

③ **完整步骤**：

> **危险操作提醒（先读再做）**：本组涉及 `chmod 777`（所有人可写）和 `sudo rm -rf`（递归强删）。只在**专用 Ubuntu 虚拟机**里做，不碰真实系统；动手前先 `pwd` 和 `ls` 核对目标；实验中建的 /tmp/openbox、/tmp/stickybox 只允许删这两个目录，**绝不能把 rm -rf 指向 /tmp 本身或其他路径**；做完按④的收尾恢复现场。

```
$ sudo mkdir /tmp/openbox; sudo chmod 777 /tmp/openbox
$ sudo touch /tmp/openbox/rootfile      # root 的文件
$ sudo -u alice sh -c 'whoami; rm /tmp/openbox/rootfile' # 显示 alice，并删除 root 的文件
# 对照实验：在独立 sticky 目录中验证，不修改系统 /tmp 权限
$ sudo mkdir -p /tmp/stickybox; sudo chmod 1777 /tmp/stickybox
$ sudo touch /tmp/stickybox/rootfile2
$ sudo -u alice sh -c 'whoami; rm /tmp/stickybox/rootfile2' # 显示 alice，然后 rm: cannot
remove '…/rootfile2': Operation not permitted
$ sudo rm -rf /tmp/stickybox
```

④ **怎么验证做对了**：目录 777（所有人可写）时，alice 删掉了 root 的文件——推演与实验一致。结论一句话：**文件能不能删，由目录的写权限决定；粘滞位（t）是这条规则的唯一刹车。** 收尾（练习结束恢复现场）：sudo rm -rf /tmp/openbox; sudo userdel -r alice; sudo userdel -r bob（userdel 一次只能删一个用户；报 No such user = 已删或没建过，属正常）

阶段 4 答案

【基础 4-1 ★】进程定位

① **先想什么**：程序跑起来就是进程，每个进程有个身份证号。

② **为什么用这个命令**：PID（Process ID）是系统给每个进程的唯一编号，操作进程（kill 等）全靠它。`sleep 300 &` 末尾的 & 表示"放后台跑"。

③ **完整步骤**：

```
$ sleep 300 &
SLEEP_PID=$!              # $! = 刚启动后台任务的实际 PID（别用 5432 这种写死数字）
$ ps -p $SLEEP_PID            # 先核对：确实是 sleep 300
$ ps -ef | grep sleep     # 或 ps aux | grep sleep，也能找到
```

④ **怎么验证做对了**：ps 输出里 sleep 那一行的 PID 与 $SLEEP_PID 一致。结束它：`kill $SLEEP_PID`。

【基础 4-2 ☆】快速定位

① 先想什么：只知道进程名，不想翻 ps 全表。

② 为什么用这个命令：`ps aux | grep xxx` 是"全表 + 过滤"两步走；**`pgrep xxx`** 一步到位，直接吐 PID。pgrep 还能加条件（如 `pgrep -u alice` 按用户）。

③ 完整步骤：

```
$ sleep 300 &
SLEEP_PID=$!                # $! = 刚启动的后台任务真实 PID
$ ps aux | grep sleep         # 能看到 PID 和整行信息
$ pgrep sleep                 # 可能输出多个 PID：同名进程都会列出（以你实际为准）
$ pgrep -l sleep              # -l 连名字一起显示
$ kill $SLEEP_PID             # 实验完清理（先 ps -p $SLEEP_PID 确认再 kill）
```

④ 怎么验证做对了：两种方式得到的 PID 相同。用法定位：**要看详情用 ps|grep，只要编号用 pgrep，脚本里自动处理用 pgrep 更方便。**

【基础 4-3 ★】优雅 vs 强杀

① 先想什么：kill 不是"一刀捅死"，它发的是"信号"，默认信号是"请你退出"。要让实验可见，需要一个小程序：它能"收到信号后写一行清理日志"。

② 为什么用这个命令：**`kill PID`**（PID 用 $! 保存实际值，见 ③；也可用 4-1 学的 ps aux | grep 或 pgrep 查，但可能匹配多个同名进程）发 SIGTERM——程序可以"打扫房间"（写日志、关文件）后体面退出；**`kill -9 PID`** 发 SIGKILL——系统直接处决，程序来不及做任何清理。所以能优雅就优雅。trap（捕获信号）就是程序安装的"信号处理器"。

③ 完整步骤（先写脚本，再动手实验）：

```
$ cat > run.sh <<'EOF'
#!/bin/bash
# run.sh - 练习用：收到退出信号时会写一行清理记录
trap 'echo "$(date +%T) 收到退出信号，清理中..." >> run.log; exit 0' TERM
while true; do
  sleep 1
done
EOF
$ chmod +x run.sh
$ ./run.sh &                  # 后台跑
RUN_PID=$!                     # $! = 刚才这个后台任务的实际 PID
$ ps -p $RUN_PID               # 先核对：确实是 ./run.sh（有无关同名进程也不会误杀）
$ kill $RUN_PID                # 优雅终止
$ cat run.log                 # 能看到"收到退出信号，清理中..."
# 再试一次：
$ ./run.sh &
RUN_PID=$!
$ ps -p $RUN_PID && kill -9 $RUN_PID   # 强杀（先确认是本实验进程）
$ cat run.log                 # 没有新增清理记录
```

④ 怎么验证做对了：同一个程序，kill 后日志里有"清理中"记录，kill -9 后没有——这就是"体面退场"和"当场击毙"的区别。真实服务（数据库、推理服务）被杀 -9 可能导致数据损坏，所以 **-9 永远是最后手段。**

【基础 4-4 ☆】资源观察

① 先想什么：机器卡了，先看谁在吃 CPU、内存。

② 为什么用这个命令：top 实时刷新进程表，核心列：%CPU（CPU 占用）、%MEM（内存占用）、RES（实际内存）、PID、COMMAND。按 P 按 CPU 排序、按 M 按内存排序。htop 是彩色增强版（需安装）。

③ 完整步骤：

```
$ top          # 按 q 退出；观察前几行
$ sudo apt update # 新系统先刷新软件列表（4-7 会细讲；若报 Unable to locate，多半是没 update）
$ sudo apt install htop && htop
```

④ 怎么验证做对了：能指出当前 %CPU 最高的进程是谁、load average（1/5/15 分钟平均负载）三个数字在什么位置。造一个吃 CPU 的进程验证：`yes > /dev/null &` 跑起来后 top 里它应该冲到榜首——看完记得结束它：`kill %1`（在 `jobs` 里看编号；不确定就直接 `pkill yes`）。

【基础 4-5 ★】磁盘视角

① 先想什么：df 和 du 都跟"空间"有关，但一个看分区、一个看目录。

② 为什么用这个命令：`df -h`（disk free）看每个**挂载分区**的总量/已用/剩余；`du -sh 目录`（disk usage）看某个**目录**占了多少。排障时先用 df 定位"哪个分区满了"，再用 du 下钻"谁占的"。

③ 完整步骤：

```
$ df -h               # / 分区使用率多少？
$ du -sh ~/*          # 家目录下每个东西各占多少
$ du -h --max-depth=1 ~   # 只看一层，不淹没在细节里
```

④ 怎么验证做对了：能一句话说清区别——**df 是"从天上看的全局视角"，du 是"钻进去找元凶的局部视角"**。df 是"按整个文件系统"统计（含元数据、ext4 的 5% 保留块、被删但仍有进程占着的文件等），du 只统计目录里"看得见"的文件——两边的数对不上是正常的，不用慌。

【基础 4-6 ★ 】服务管理

① **先想什么**：服务的本质是"常驻后台的进程"，systemd 是负责看管它们的大管家。

② **为什么用这个命令**：`systemctl` 是管家的总开关：status（看状态）、start（启动）、stop（停止）、restart（重启）、enable（开机自启）、disable（取消自启）。

③ **完整步骤**：

> **停 SSH 前必看**：`systemctl stop ssh` 会把远程登录服务关掉——**如果你正用 SSH 远程连到这台机器，执行后你会立刻被踢下线、再也连不回来！**只用本地终端（或虚拟机的直接窗口）做本题，别在 SSH 会话里做；做完必须 `sudo systemctl start ssh` 恢复。

```
$ systemctl status ssh        # 看 SSH 服务：active (running) 就是活着（刚装完可能先是
inactive (dead)：26.04 用 ssh.socket 激活，连一次或 sudo systemctl start ssh 就会起来）
$ sudo systemctl restart ssh  # 重启它
$ sudo systemctl stop ssh
$ systemctl status ssh        # inactive (dead)
$ sudo systemctl start ssh     # 恢复！别把练习机搞到连不上
```

④ **怎么验证做对了**：每个动作后 status 的状态都会变（running ↔ dead），最后 start 回 running。记住四连：**改配置 → restart；查问题 → status；开机自启 → enable。**平板 Termux 没有 systemd，这题标 只能在电脑做。

【基础 4-7 ★】包管理

① **先想什么**：装软件不用去官网下载，系统有"软件仓库 + 安装工具"。

② **为什么用这个命令**：`apt` 是 Ubuntu 的包管理器：`apt search 关键词`（搜）、`apt install 包名`（装）、`apt remove`（卸）、`apt update`（刷新软件列表，装之前先跑）。

③ **完整步骤**：

```
$ apt search htop           # 找到包名
$ sudo apt update
$ sudo apt install htop
$ apt list --installed | grep htop    # 确认已装
$ htop                      # 能跑起来 = 装好了
```

④ **怎么验证做对了**：`which htop` 给出路径、命令能运行。经典报错预警：忘了 `apt update` 就 install，可能报 `Unable to locate package`——先 update 再 install 的顺序要记住。

【基础 4-8 ☆】前后台机制

① **先想什么**：Ctrl+C、Ctrl+Z、"&" 都在"停进程"，但停法完全不同。

② **为什么用这个命令**：Ctrl+C 发中断信号=**结束**；Ctrl+Z=**挂起**（暂停，还活着，可复活）；jobs 看后台任务；fg 拉回前台、bg 让它在后台继续跑；`cmd &` 一开始就放后台。

③ **完整步骤**：

```
$ sleep 300          # 按 Ctrl+Z → 出现 [1]+ Stopped
$ jobs               # 看到任务1挂着
$ bg                 # 让它后台继续睡
$ fg                 # 拉回前台；再按 Ctrl+C 结束它
```

④ **怎么验证做对了**：体验过三种停法并说得出区别：**Ctrl+C=杀掉；Ctrl+Z=暂停可复活；&/bg=转到后台继续。**最实用的一招：前台跑着的长任务按 Ctrl+Z 再 bg，立刻腾出终端，任务还不中断。

阶段 5 答案

【基础 5-1 ★】按名查找

① **先想什么**：文件散在各处，按名字找——find 是"条件搜索"，grep 是"内容搜索"，分工不同。

② **为什么用这个命令**：`find 起点 -name 模式`，在起点目录下递归找匹配名字的文件。模式里的通配符要用引号包住（见 5-8）。

③ **完整步骤**：

先准备测试材料（与查找目标对口）：`mkdir -p ~/lab/findtest && touch ~/lab/findtest/a.log ~/lab/findtest/b.LOG ~/lab/findtest/c.txt`

```
$ find ~/lab/findtest -name "*.log" # 找 log 文件：只应命中 a.log（b.LOG 大小写不同不命中）
$ find ~/lab/findtest -iname "*.log" # 忽略大小写：a.log、b.LOG 都命中
$ rm -rf ~/lab/findtest   # 清理：实验产物用完就删
```

④ **怎么验证做对了**：输出全部以 .log 结尾的完整路径。对比 `find ~ -iname "*.log"`（i=忽略大小写，能多找出 .LOG）。

【基础 5-2 ☆】按属性查找

① **先想什么**：名字不知道，但知道"很大"或"最近改过"。

② **为什么用这个命令**：`-size +100M` 按大小（+大于 / -小于）；`-mtime -1` 按修改时间（-1 = 最近 1 天内，+7 = 超过 7 天）。

③ **完整步骤**：

```
$ cd ~/lab                    # 必做：先限定在练习目录
$ find . -type f -size +100M -print
$ find . -type f -mtime -1    # 最近一天改过的普通文件
$ sudo find / -xdev -type f -size +100M -print # 扩展：可能耗时并显示权限错误
```

④ **怎么验证做对了**：必做范围只看 `~/lab`，结果稳定且不会误扫系统；全盘扩展项可能耗时并遇到权限错误。若使用 `2>/dev/null`，要明确它会隐藏错误，不能把“没有输出”误判为“系统没有大文件”。

【基础 5-3 ★】查找+执行

① **先想什么**：找到一批文件后要对它们统一动手——把 find 和"执行"接起来。

② **为什么用这个命令**：`find ... -delete`（内置删除）或 `-exec rm {} \;`（对每个找到的文件执行 rm）。危险点：条件写错会误删，所以动手前先用不带 -delete 的 find 看一遍清单。

③ **完整步骤**：

```
$ touch ~/lab1/junk1.tmp ~/lab1/junk2.tmp
$ find ~/lab1 -name "*.tmp"           # 先看清单！两个文件
$ find ~/lab1 -name "*.tmp" -delete   # 确认无误再删
$ ls ~/lab1                           # junk 没了
```

④ **怎么验证做对了**：删除前先预览、删除后消失。养成铁律：**find -delete / -exec rm 之前，必先跑一次不带删除的 find 核对名单。**

【基础 5-4 ★】排序去重

① **先想什么**：uniq 只能合并"相邻"的重复行，所以先 sort 让相同的行挨在一起。

② **为什么用这个命令**：`sort` 排序；`uniq` 去重（只对相邻行生效）；`uniq -c` 在每行前面加上出现次数。顺序：sort → uniq -c 就是"统计每个值出现几次"的标准套路。

③ **完整步骤**：

```
$ printf 'b\na\na\nb\nc\n' > names.txt
$ sort names.txt | uniq -c
2 a
2 b
1 c
```

④ **怎么验证做对了**：输出"次数 + 值"，a、b 各 2 次。反例体会：直接 `uniq names.txt`（不 sort）只会去掉相邻重复，输出 b/a/b/c——这就是"先 sort 再 uniq"的原因。

【基础 5-5 ☆】日志统计套路

① **先想什么**：从"提取字段 → 排序 → 计数 → 取前几"是日志分析的标准流水线。

② **为什么用这个命令**：`awk '{print $1}'` 取第一列（如 IP）；`sort` 让相同值相邻；`uniq -c` 计数；`sort -rn` 按次数倒序；`head -5` 取前五。五个小工具接力，一条管道完成。

③ **完整步骤**：

```
$ rm -f ~/lab/app.log # 重复运行前先重建：清掉旧日志，状态可控
$ echo '2026-08-01 ERROR disk fail' >> ~/lab/app.log
$ echo '2026-08-01 ERROR disk fail' >> ~/lab/app.log
$ echo '2026-08-01 WARN retry' >> ~/lab/app.log
$ echo '2026-08-02 ERROR net down' >> ~/lab/app.log
$ awk '{print $1}' ~/lab/app.log | sort | uniq -c | sort -rn | head -5
3 2026-08-01
 1 2026-08-02
 ...（示例—你的数字以实际写入为准；数字最大的是"发日志最多"的来源）
```

④ **怎么验证做对了**：输出按数字从大到小排前 5。这个套路可以替换任何"列"和任何"文件"——以后统计访问日志 Top IP、Top URL 用的就是它。试着把 $1 换成别的列再跑一次。

【基础 5-6 ★】打包压缩

① **先想什么**：打包（归档）和压缩是两件事：tar 把多个文件合成一个，gzip 把体积压小。

② **为什么用这个命令**：`tar -cf 包名 目录` 只打包不压缩；`tar -czf 包名.tar.gz 目录` 打包 + gzip 压缩（z = gzip）。

③ **完整步骤**：

```
$ cd ~ # 在家目录下执行（别在 lab1 里打包，否则解包对比会乱）
$ tar -cf lab1.tar ~/lab1
$ tar -czf lab1.tar.gz ~/lab1
$ ls -lh lab1.tar lab1.tar.gz   # .gz 通常更小（文本类明显）；若材料是视频/已压缩文件，可能几乎不变小—属正常，别当实验失败
```

跑 tar 时终端还会多出一行 `tar: Removing leading `/' from member names`——那是 tar 提示它把包内路径开头的 / 去掉了（包内用相对路径更安全）。**属正常提示，不是报错**，不影响打包与解包。

④ **怎么验证做对了**：两个产物都在，用 ls -lh 实际对比体积（文本类通常 .tar.gz 更小，已压缩数据可能持平）。口诀：**打包 czvf、解包 xzvf**（c=创建、x=解包、z=gzip、v=显示过程、f=指定文件）。

【基础 5-7 ☆】解包验证

① **先想什么**：备份的价值在于"还原后一模一样"，要用 diff 证明。

② **为什么用这个命令**：`tar -xzf` 解包到指定目录（-C）；`diff -r` 递归对比两个目录，没有任何输出 = 完全一致。

③ **完整步骤**：

```
$ tar -tzf ~/lab1.tar.gz | head -3   # 先查包里记录的原路径（head 只看前几行）
$ rm -rf ~/restore-check && mkdir -p ~/restore-check   # 重复运行前清旧解包目录
$ tar -xzf ~/lab1.tar.gz -C ~/restore-check
$ RESTORED_DIR="$(find ~/restore-check -type d -path '*/lab1' -print -quit)"   # 自动找到解出来的 lab1 目录，不写死用户名
$ if [ -z "$RESTORED_DIR" ]; then echo "没找到解包出来的 lab1，检查上一步 tar 是否成功"; exit 1; fi
$ diff -r "$HOME/lab1" "$RESTORED_DIR"   # 无输出 = 一致
```

④ **怎么验证做对了**：diff 静默（无任何输出）即验证通过；如果有差异，diff 会列出哪些文件不同。找不到目录时脚本会提示并退出，而不是报一长串 diff 错误。

【基础 5-8 ★】引号原理

① **先想什么**：通配符谁负责展开？是 shell 还是 find？引号决定了答案。

② **为什么用这个命令**：不加引号时，**shell 先**把 *.log 展开成当前目录里匹配的文件名，再传给 find——结果取决于当前目录有什么；加了引号，*.log 原样传给 find，由 **find 自己**去每个子目录匹配。后者才是你想要的。

③ **完整步骤**：

```
$ mkdir -p ~/lab/empty && cd ~/lab/empty   # 先建一个没有 .log 文件的目录并进去
$ find . -name *.log          # shell 找不到可展开的 → 原样传，碰巧能用（无输出、rc=0）
$ touch a.log b.log
$ find . -name *.log          # 现在被展开成 find . -name a.log b.log → 直接报错
find: paths must precede expression: `b.log'
find: possible unquoted pattern after predicate `-name'?
$ find . -name "*.log"        # 带引号，行为永远稳定（a.log、b.log 都能命中）
$ rm a.log b.log && cd ~      # 清理实验产物，回原目录
```

④ **怎么验证做对了**：同一个命令，目录里有没有 .log 文件时行为不同——这就是"shell 抢先展开"的证据。结论：**find 的通配符永远加引号**，把解释权留给 find。

附录 A · 本册命令速查

阶段	命令一览
阶段 1	`pwd` · `ls` · `cd` · `mkdir(-p)` · `touch` · `cp(-r)` · `mv` · `rm(-r/-i)` · `rmdir` · `echo` · 花括号展开 `{1..5}`
阶段 2	`cat` · `less` · `head(-n)` · `tail(-n/-f)` · `grep(-i/-v/-c/-r/-n)` · `wc(-l)` · 管道（`|`） · 重定向 `> >> 2>/dev/null` · `seq`（造材料）· `printf`（造材料，同 `seq`）
阶段 3	`ls -l(-ld)` · `chmod(符号法/数字法/2770)` · `chown` · `chgrp` · `stat(-c %a)` · `sudo` · `useradd(-m)` · `passwd` · `usermod(-aG)` · `groupadd` · `su`（本册用 `sudo -u` 代替；见 01 册 5.1） · `id` · `userdel(-r)`
阶段 4	`ps(-ef/aux)` · `pgrep(-l)` · `top` · `htop` · `kill` · `kill -9` · `jobs` · `fg` · `bg` · `Ctrl+Z` / `Ctrl+C` · `df(-h)` · `du(-sh/--max-depth)` · `systemctl(status/start/stop/restart/enable)` · `apt(search/update/install/list/remove)`
阶段 5	`find(-name/-iname/-size/-mtime/-delete/-exec)` · `file`（判断类型） · `sort(-r/-n)` · `uniq(-c)` · `awk '{print $1}'` · `tar(-cf/-czf/-xzf/-C)` · `gzip` · `diff(-r)` · `head -n 5`

附录 B · 术语表

术语	解释
目录树	Linux 的所有文件和目录组成一棵以 / 为根的树
绝对路径 / 相对路径	从 / 出发 vs 从当前目录出发描述位置
家目录（~）	每个用户自己的地盘，/home/用户名
权限位	ls -l 第一列的 9 位：属主/属组/其他人 × 读写执行
属主 / 属组	文件的所有者 / 文件所属的用户组
标准输入 / 标准输出	程序默认的"入口"（键盘）和"出口"（屏幕）
管道（|）	把左边命令的标准输出接到右边命令的标准输入
重定向（> / >>）	把输出写进文件：覆盖 / 追加
进程 / PID	运行中的程序 / 它的唯一编号
信号	发给进程的控制消息：TERM=请你退出，KILL=强制处决
前台 / 后台	占用终端运行 / 不占用终端运行
挂载 / 分区	把磁盘区域接入目录树 / 磁盘上划分的区域
包管理器（apt）	从软件仓库安装/卸载软件的工具
通配符（* ?）	匹配文件名的符号：任意多个字符 / 恰好一个字符
归档 / 压缩（tar）	多个文件合成一个 / 减小体积（gzip）
粘滞位（t）	目录权限上的特殊位：只有文件属主或 root 能删除（如 /tmp）